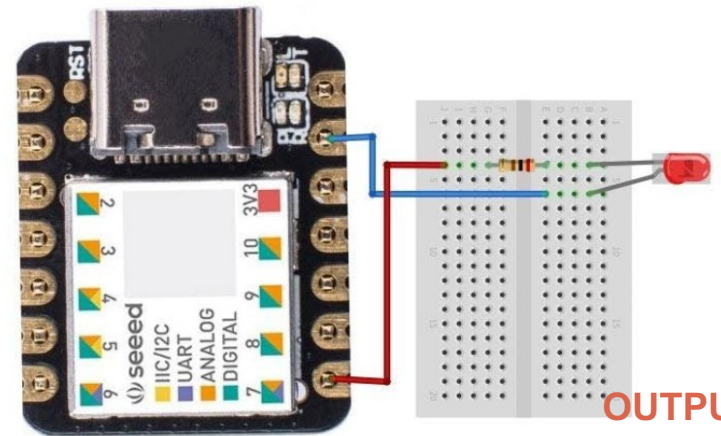


INPUT and OUTPUT

Logic levels, ADC, and PWM

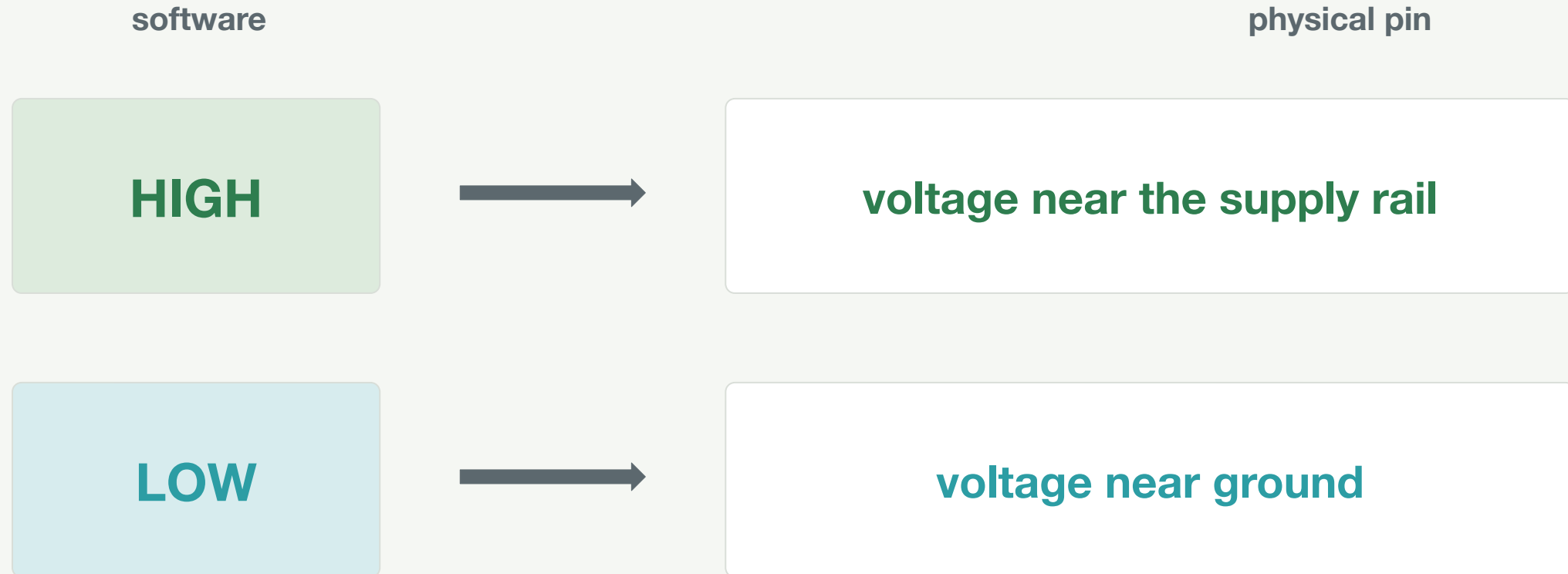


INPUT



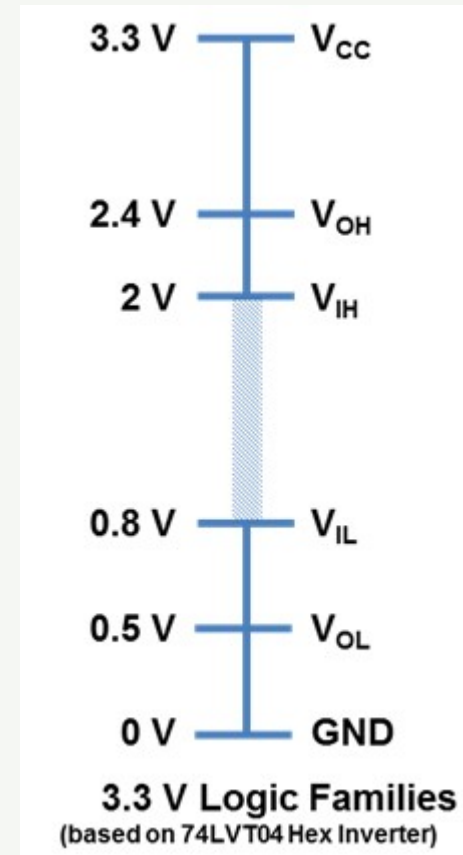
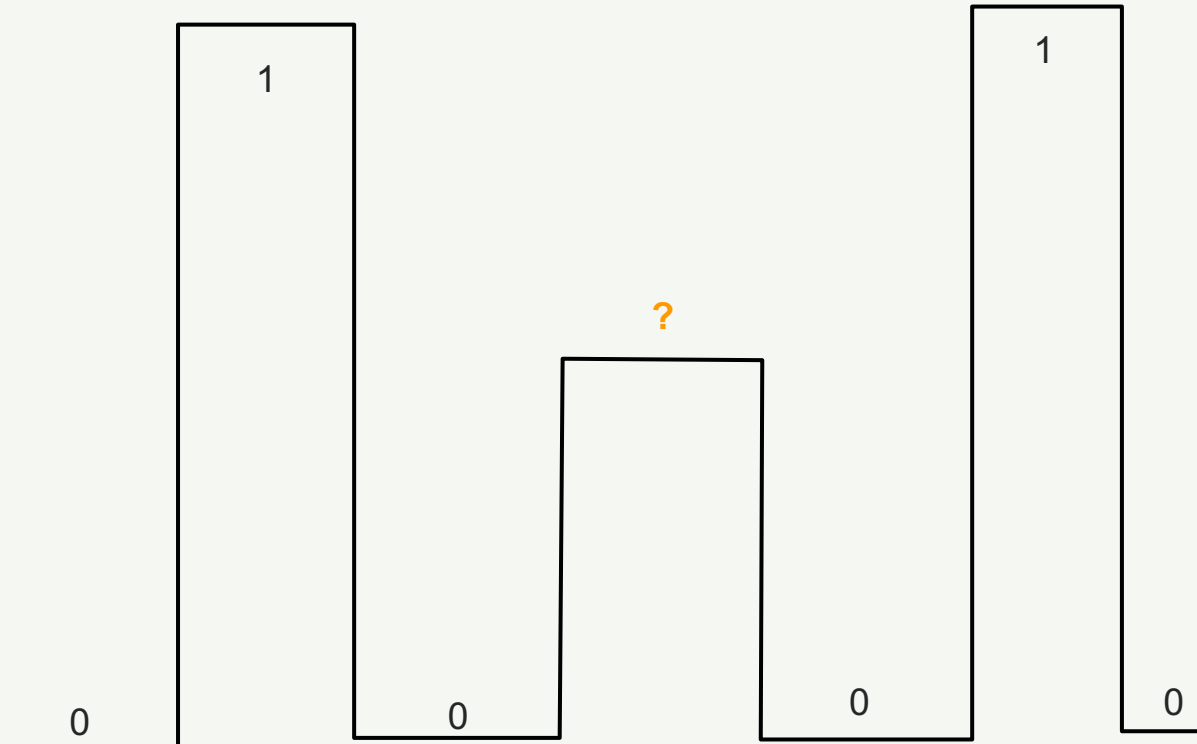
OUTPUT

Digital Communication



Digital does not mean voltage disappears; it means voltage is interpreted in categories.

Logic Levels



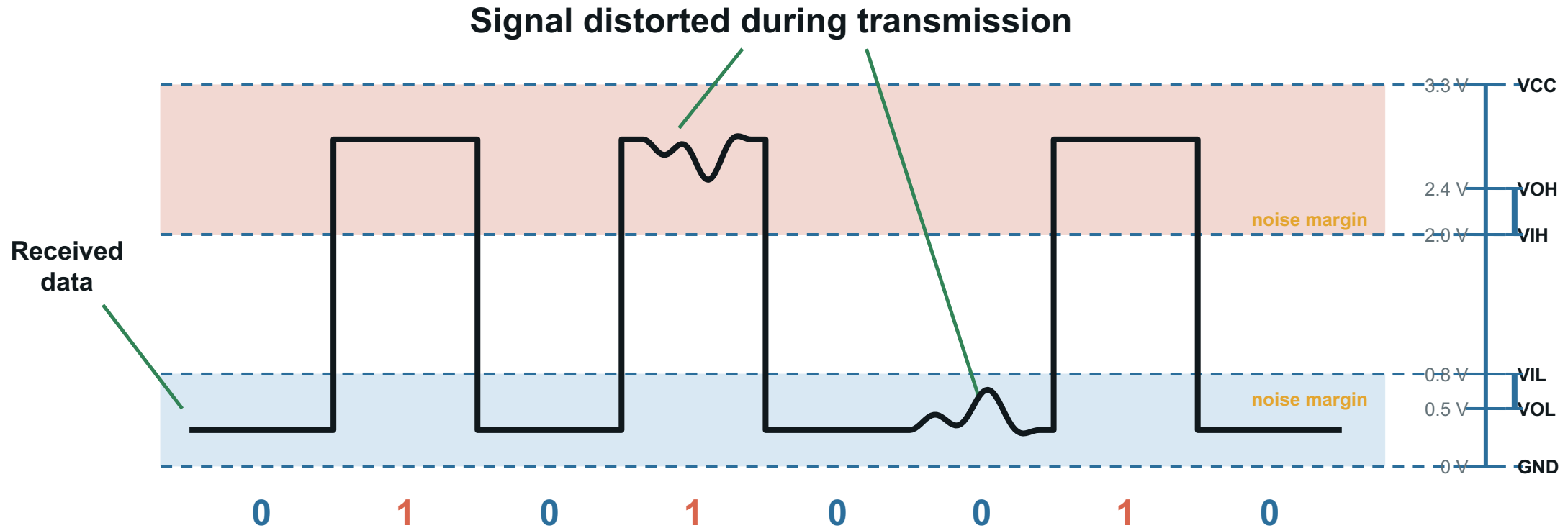
HIGH

UNDEFINED
do not design here

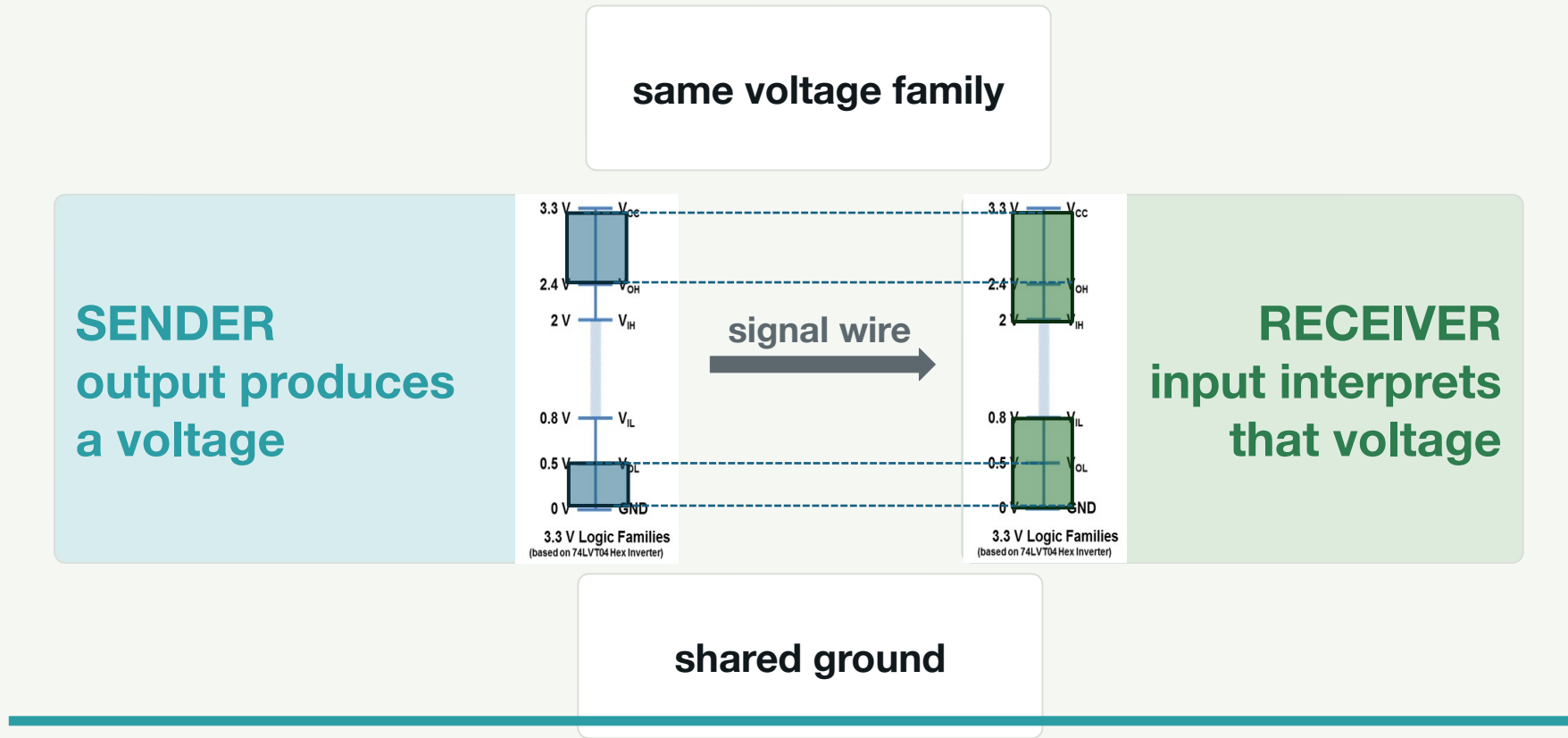
LOW

Noise Margins

A noise margin is the voltage change a valid output can tolerate before the receiver reaches its threshold.



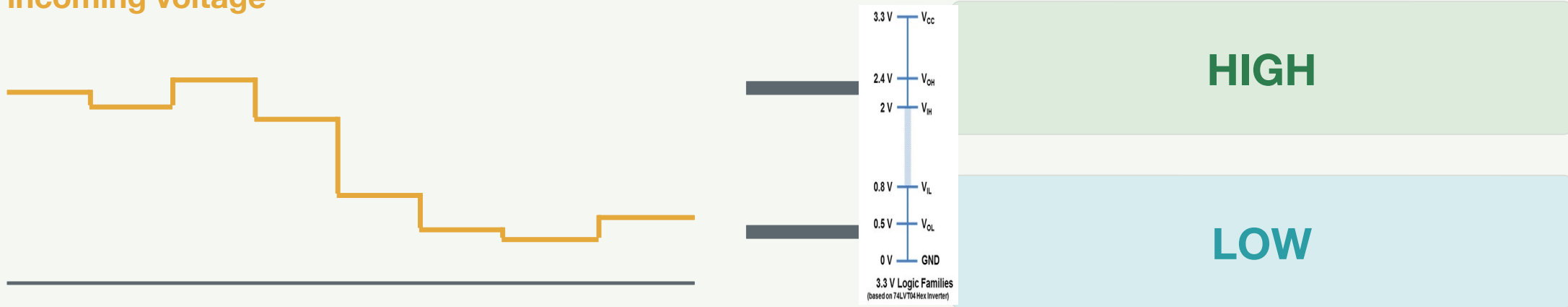
Logic-Level Compatibility



Ground gives both devices the same reference for measuring the signal voltage.

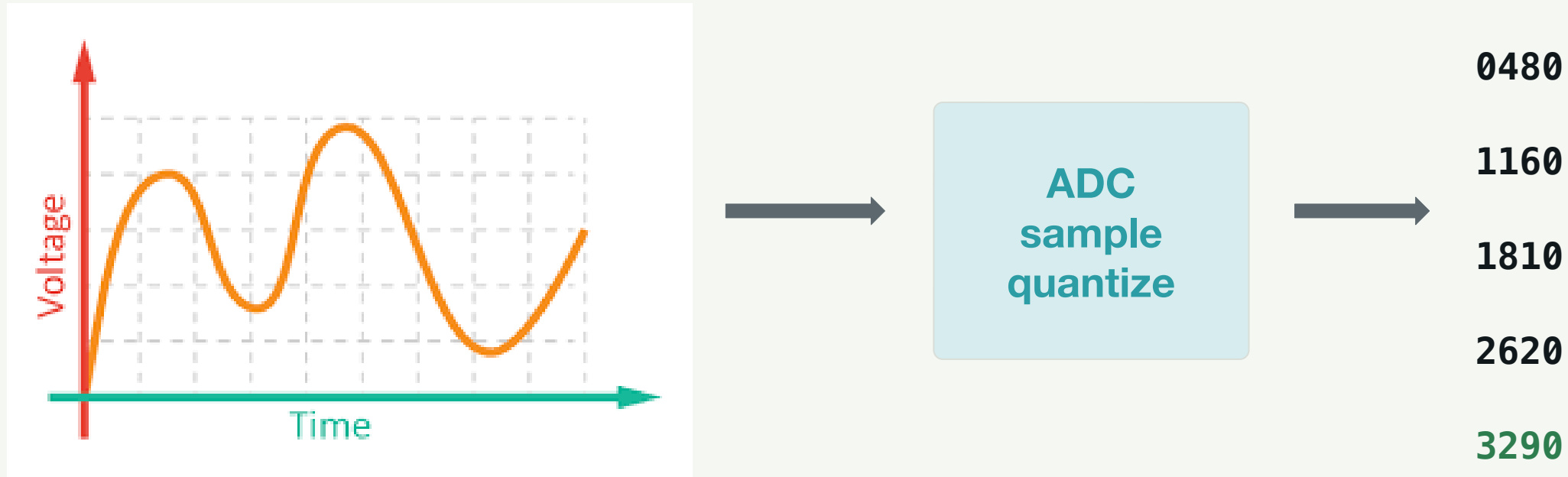
What if we want the actual voltage?

incoming voltage



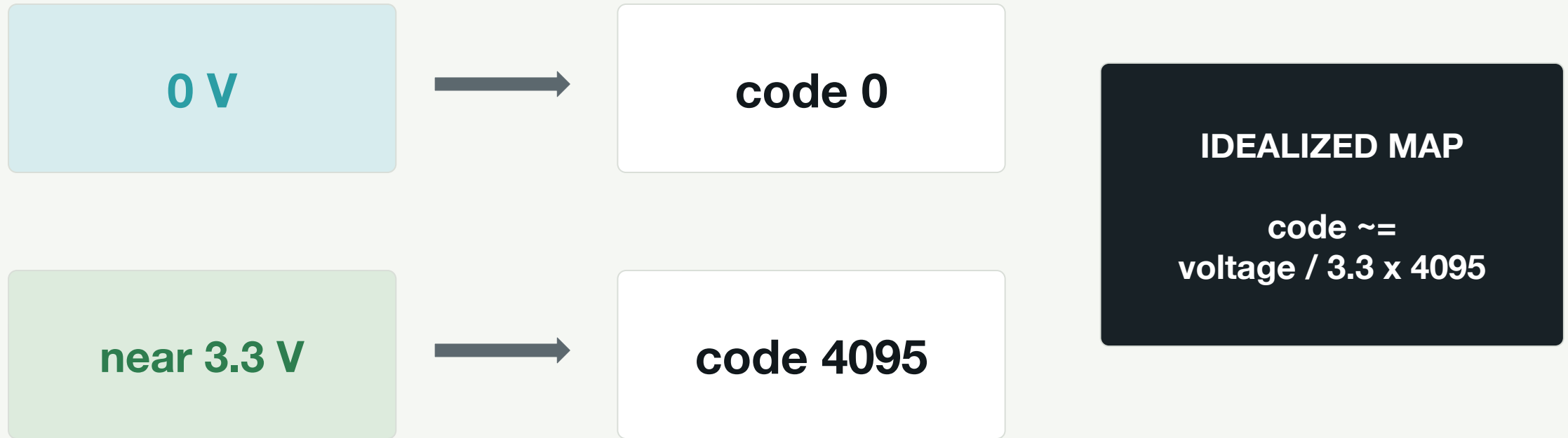
That requires an analog input and an ADC.

Analog-to-Digital Conversion (ADC)



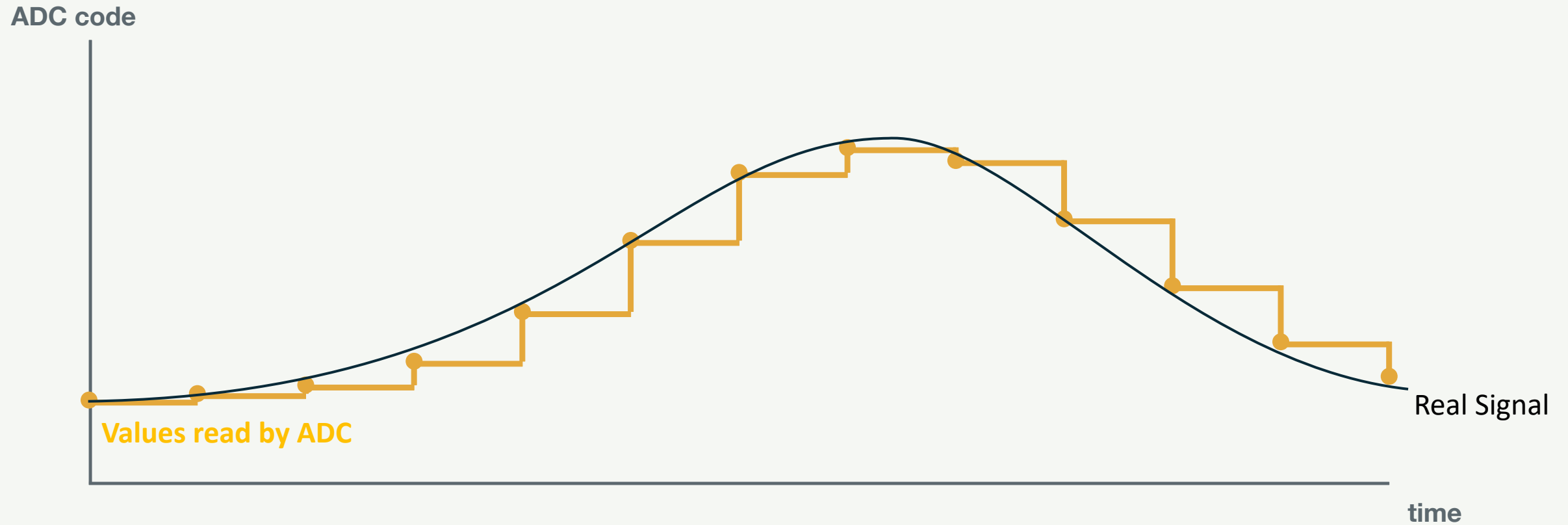
The code approximates the sampled voltage; it is not the physical quantity itself.

ADC Resolution



More bits create finer code steps, not a more accurate sensor by themselves.

Sampling Over Time



Light-Dependent Resistors

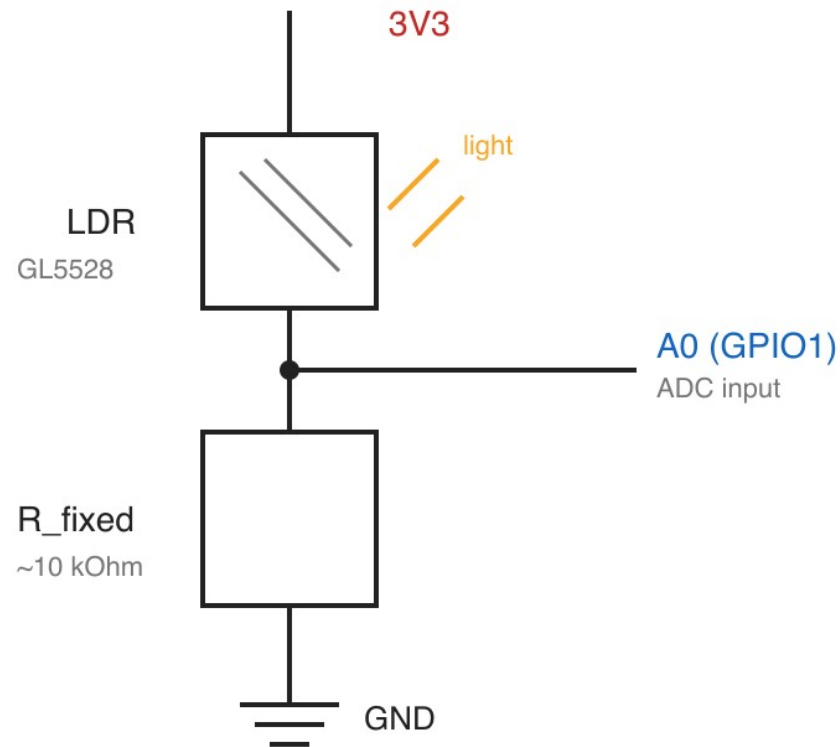


MORE LIGHT
lower resistance

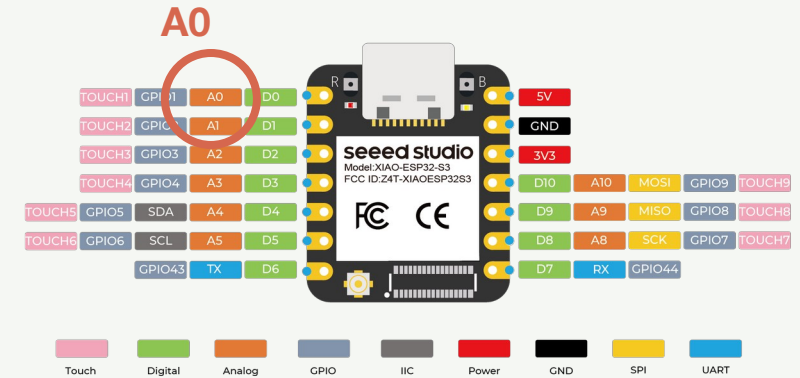
LESS LIGHT
higher resistance

Connecting an Analog Sensor

GL5528 LDR voltage divider



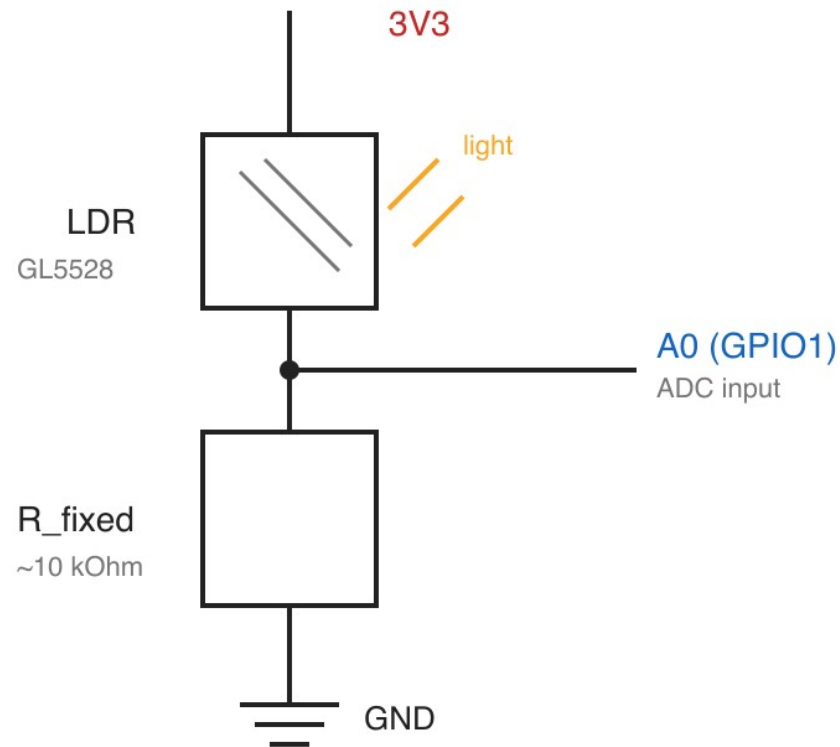
Brighter light lowers the LDR resistance, so the A0 node rises with light and falls when covered. Confirm the direction and dark level on YOUR board.



**A0 is the board label
GPIO1 is the software pin**

Voltage Dividers

GL5528 LDR voltage divider

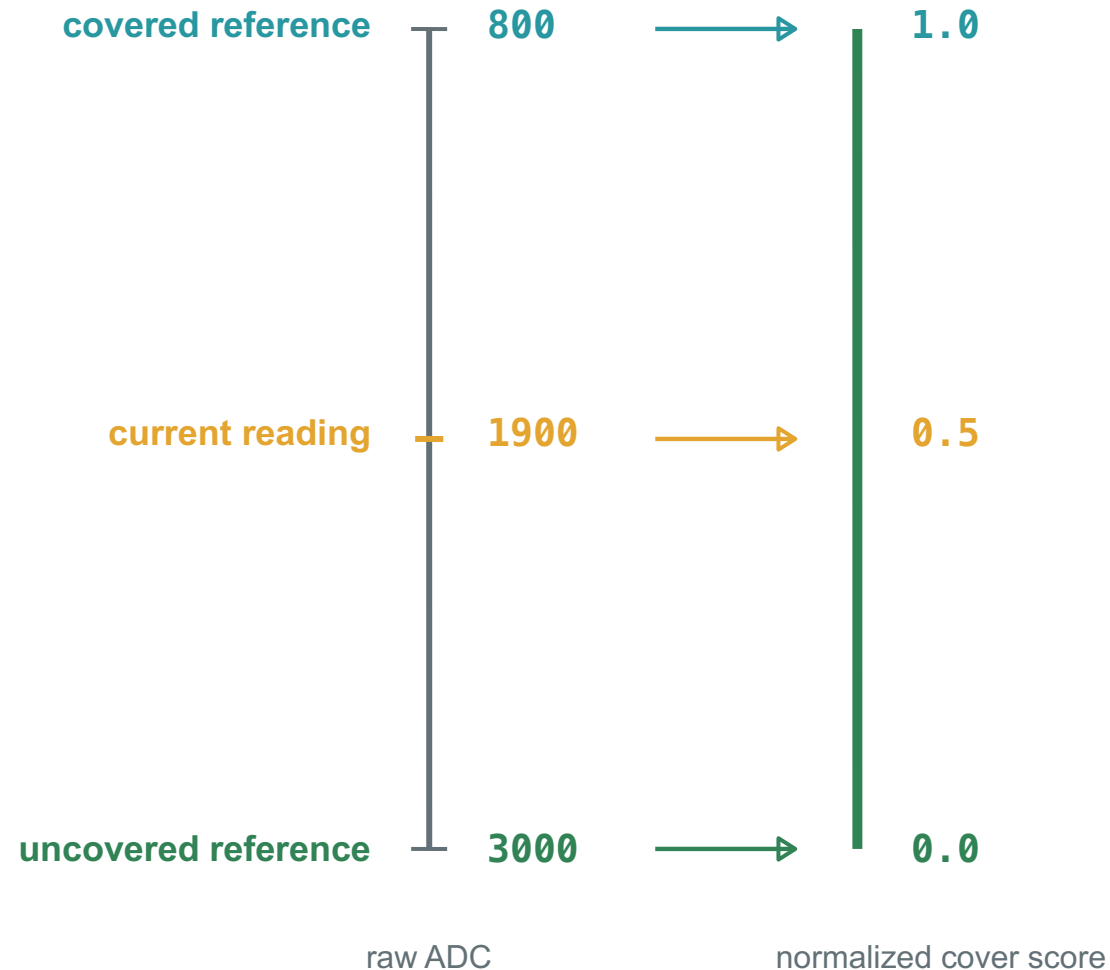


Brighter light lowers the LDR resistance, so the A0 node rises with light and falls when covered. Confirm the direction and dark level on YOUR board.

The junction voltage depends on the ratio of the two resistances.



Two-Point Calibration



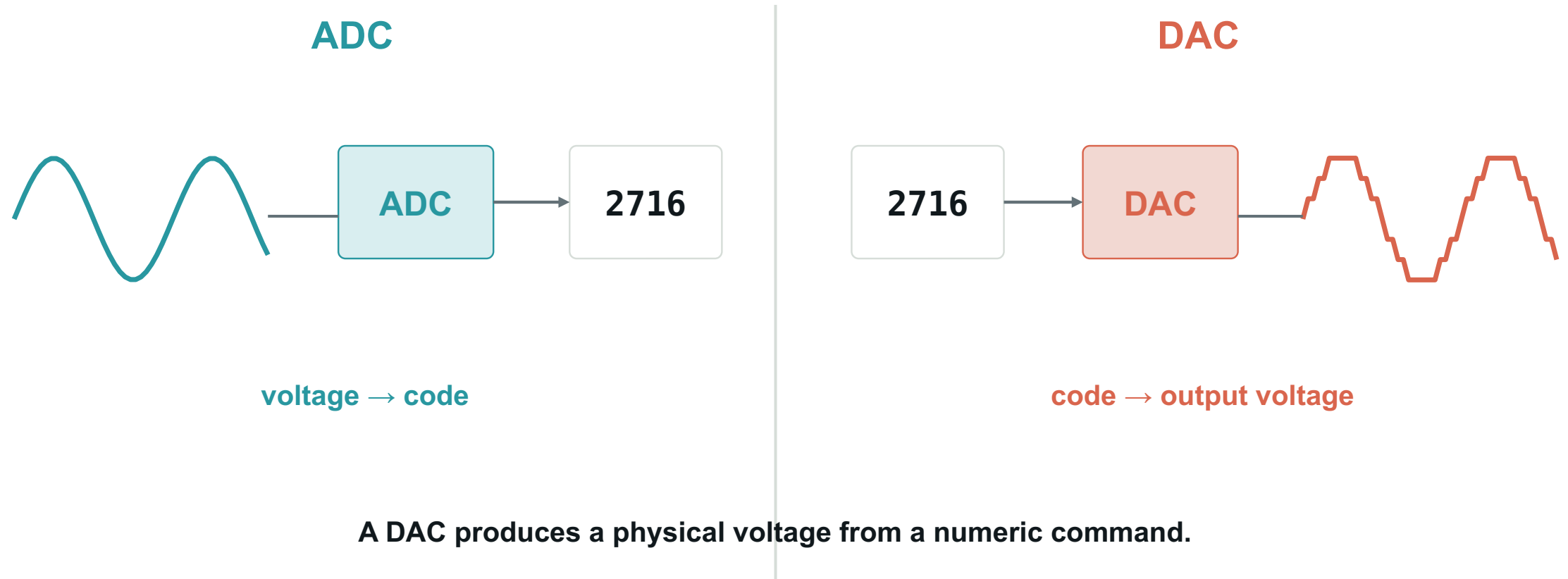
```
int span = uncovered - covered;  
float score =  
    (uncovered - reading) / float(span);  
score = constrain(score, 0.0f, 1.0f);
```

Measure the endpoints in the current environment.

Calibration defines the scale. The application chooses thresholds on that scale later.

Digital-to-Analog Conversion (DAC)

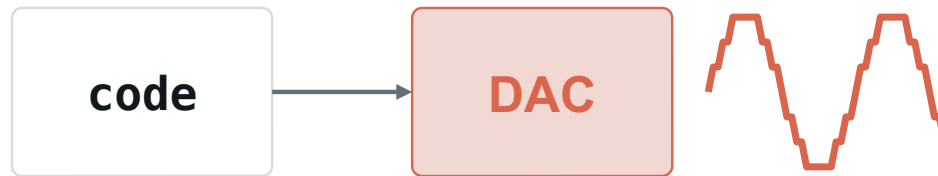
ADC and DAC move in opposite directions across the same boundary.



DAC Availability

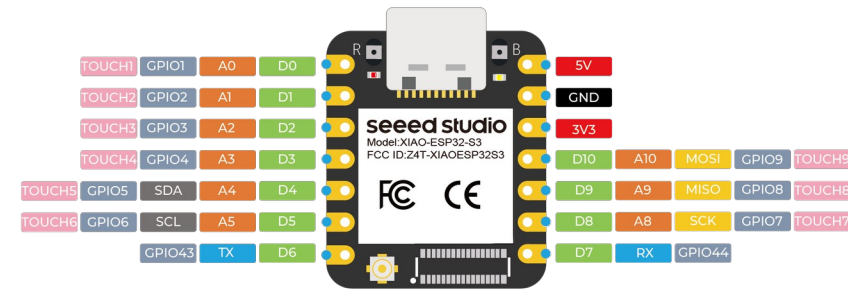
A DAC is a hardware peripheral. Whether one is available depends on the microcontroller and the selected pin.

Some microcontrollers



numeric command becomes an output voltage

XIAO ESP32-S3

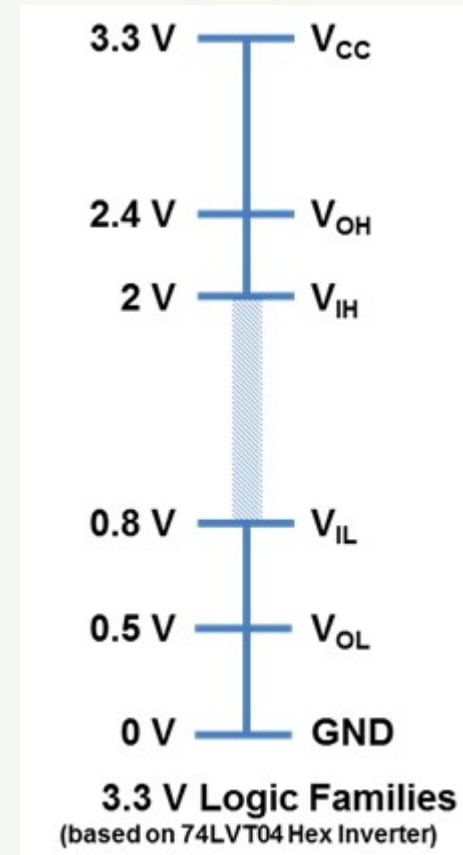
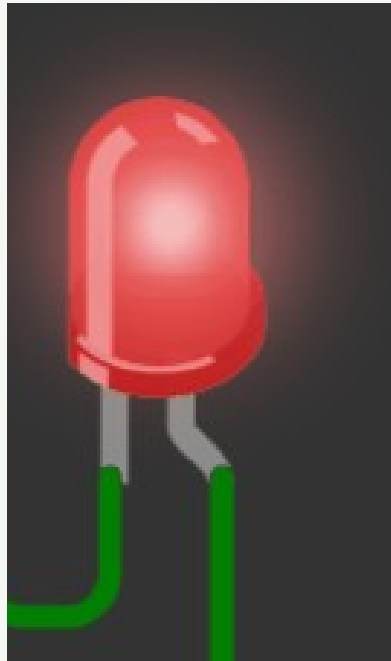


No DAC output is exposed

Use PWM when the load can respond to an average effect.

Analog-Like Output from a Digital Pin

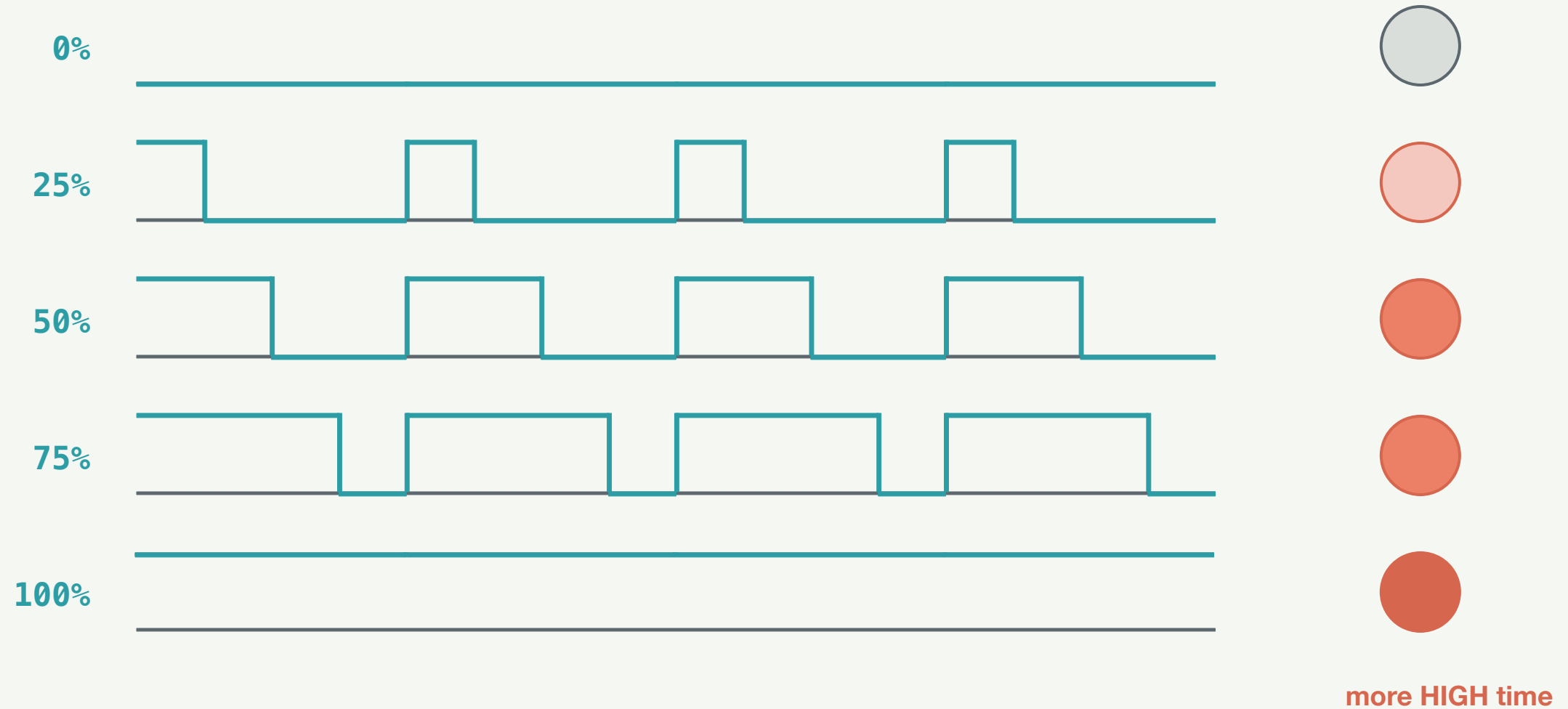
How can the same digital pin produce intermediate LED brightness?



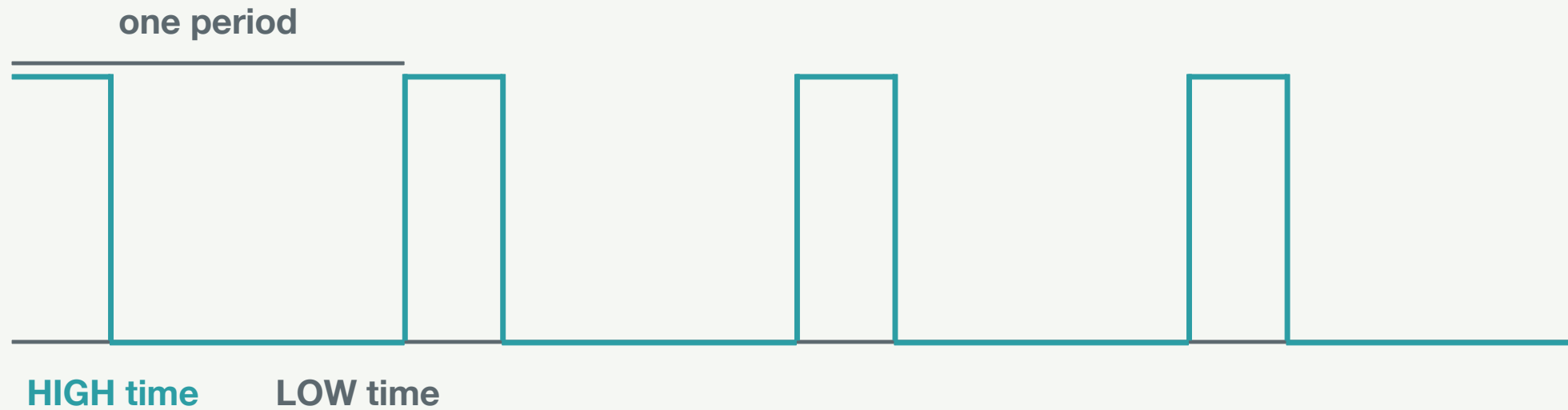
HIGH

LOW

Pulse-Width Modulation (PWM)



Pulse Width and Period



pulse width = time spent HIGH during each period

Duty Cycle

HIGH time
2 ms

/

period
8 ms

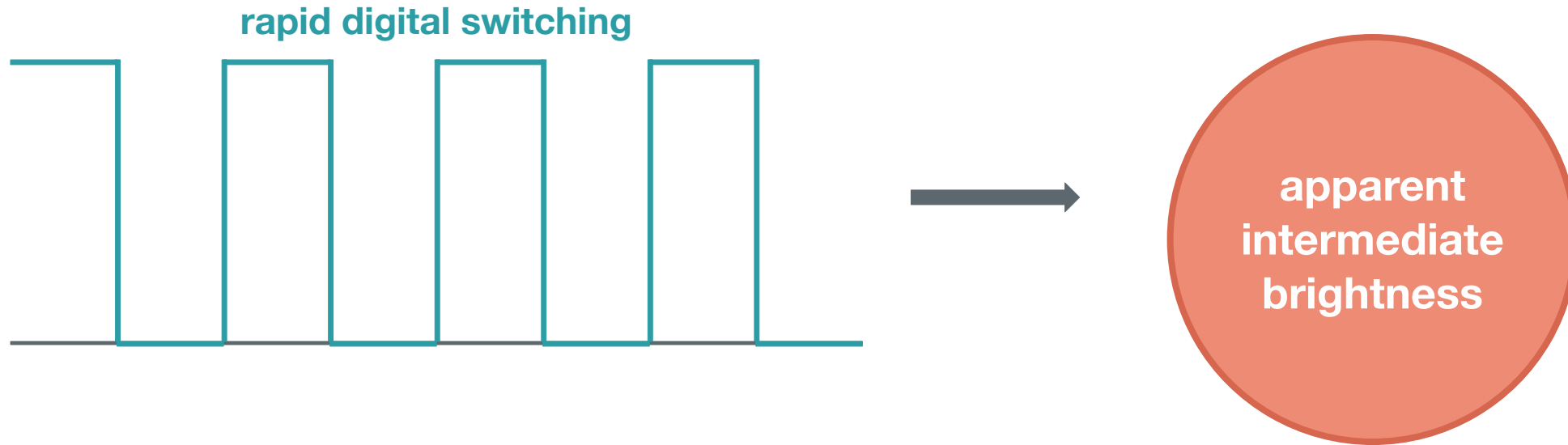
=

25%
duty



The output is still only HIGH or LOW at every instant.

Average Response to PWM

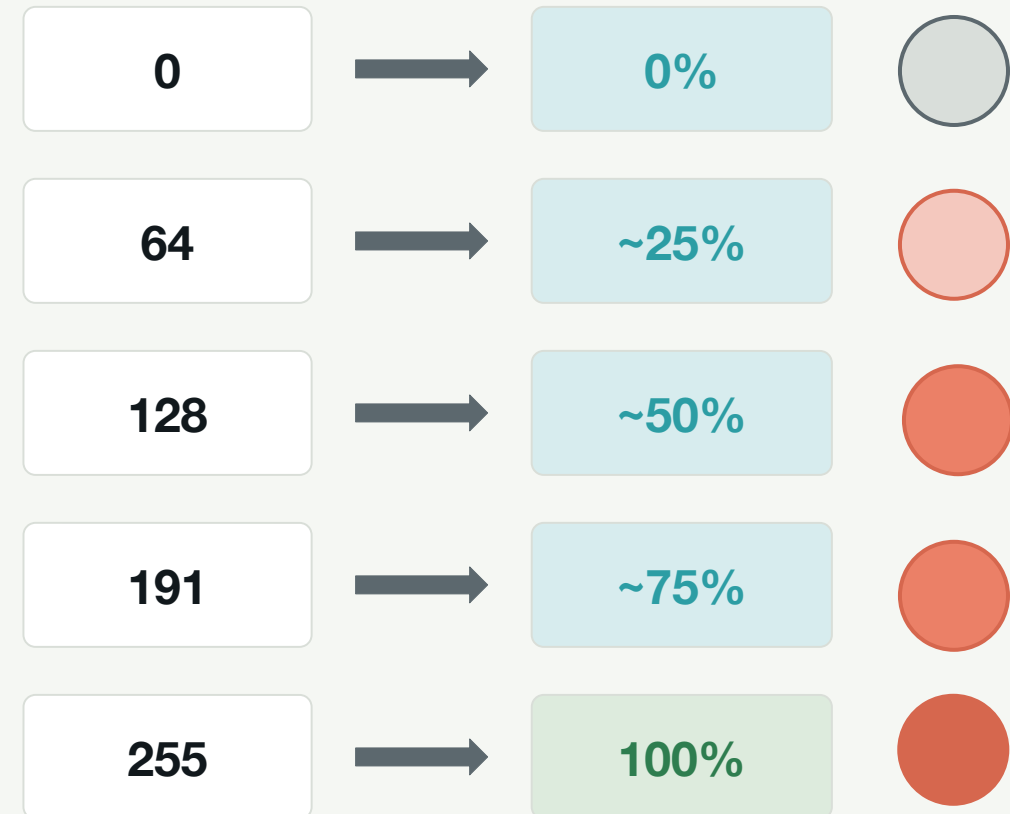


The switching must be fast relative to the load's response.

A slower load or observer responds to duty cycle rather than each transition.

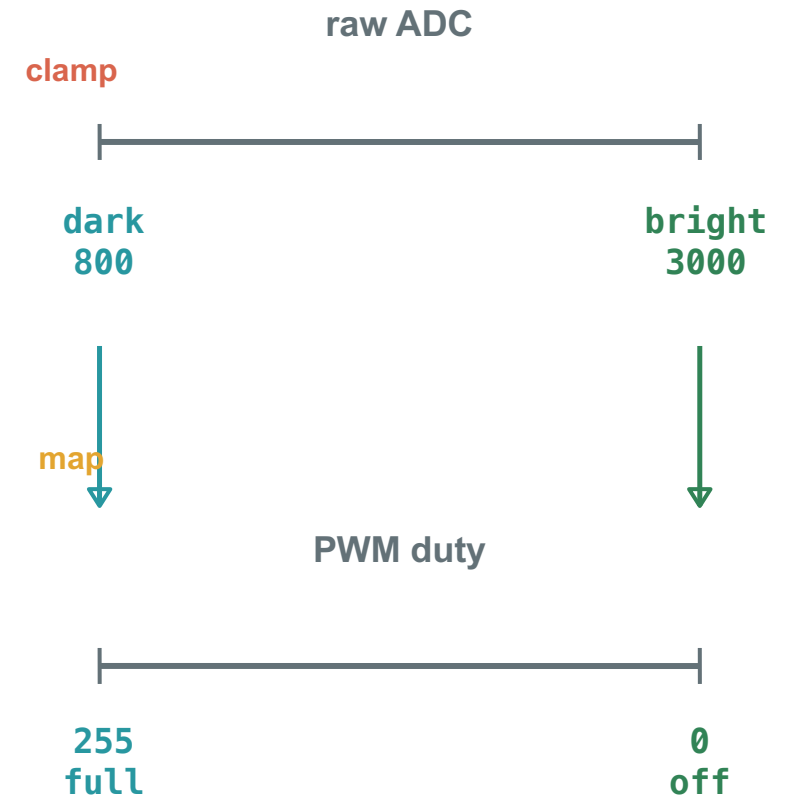
PWM Commands

```
pinMode(PWM_LED_PIN, OUTPUT);  
analogWriteResolution(PWM_LED_PIN, 8);  
  
analogWrite(PWM_LED_PIN, duty);
```



Mapping an ADC Range to PWM

```
uint8_t mapToDuty(int reading, int bright,  
                  int dark, uint8_t maxDuty) {  
    if (reading >= bright) return 0;  
    if (reading <= dark) return maxDuty;  
  
    int64_t span = bright - dark;  
    int64_t distance = bright - reading;  
    return (distance * maxDuty + span / 2) / span;  
}
```



+ span / 2 before integer division rounds to the nearest duty.